Year 2 – Data Structures & Algorithms

Project Report

RouteForge

A Graph Algorithm Visualisation System

Module: Data Structures & Algorithms

Language: Python 3.11+

Framework: pygame 2.x

Repository: RouteForge

16th May 2026

Abstract

RouteForge is an interactive graph construction and algorithm visualisation system implemented in Python. The project provides a modular architecture separating the core graph data structure, algorithmic implementations, a command-line interface, and a real-time pygame-based graphical visualiser. Five classical graph algorithms are implemented: Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's shortest-path algorithm, Prim's minimum spanning tree (MST), and Kruskal's MST. The visualiser supports step-by-step animated playback of BFS, DFS, and Dijkstra with interactive controls, enabling inspection of frontier evolution, edge relaxation, and shortest-path reconstruction. A random graph generator is included to facilitate rapid testing. This report describes the system architecture, data structures, algorithmic design, complexity analysis, and implementation decisions.

Contents

1 Introduction

Graph theory underpins a broad class of real-world problems, from network routing and social-network analysis to geographic pathfinding and dependency resolution. Understanding how fundamental graph algorithms operate is a core competency in computer science education; yet the abstract nature of queue and priority-queue mechanics, visited sets, and predecessor maps often makes comprehension difficult from code or pseudocode alone.

RouteForge addresses this gap by coupling correct, modular algorithm implementations with an interactive step-by-step visualiser. The system is designed around three principles:

(i) **Separation of concerns** – the graph data structure, the algorithms, the CLI, and the visualiser are entirely independent modules.

(ii) **Correctness first** – each algorithm is implemented faithfully to its canonical form; the visualiser replicates the algorithm logic internally rather than instrumenting or modifying the production algorithm code.

(iii) **Interactivity** – the user can construct arbitrary graphs, generate random instances, and animate algorithms at configurable speeds, stepping forwards and backwards through execution.

The remainder of this report is structured as follows. Section **??** describes the overall system architecture and module layout. Section **??** details the graph data structure and supporting types. Section **??** covers the input validation layer. Sections **??**–**??** present each algorithm with pseudocode, implementation notes, and complexity analysis. Section **??** describes the visualiser design, animation model, and interaction system. Section **??** covers the random graph generator. Section **??** describes the CLI. Section **??** concludes with reflections and potential extensions.

2 System Architecture

2.1 Module Layout

The project is organised as a Python package rooted at `src/`. The directory tree is as follows:

```
RouteForge/
 main.py
 src/
     algorithms/
        bfs.py
        dfs.py
```

```
    dijkstra.py
    helpers.py
    kruskal.py
    prim.py
core/
    graph.py
    validators.py
data/
ui/
    menu.py
utils/
    graph_generator.py
visualization/
     pygame_view.py
```

2.2 Dependency Graph

The inter-module dependency structure is intentionally acyclic and layered:

- `src/core/` has no internal dependencies; it defines the foundation types used by all other layers.
- `src/algorithms/` depends only on `src/core/`.
- `src/utils/` depends only on `src/core/`.
- `src/ui/` depends on `src/core/`, `src/algorithms/`, `src/utils/`, and `src/visualization/`.
- `src/visualization/` depends on `src/core/` and `src/core/validators`.
- `main.py` depends only on `src/ui/menu.py`.

This layering ensures that the core graph and algorithm modules can be tested and used independently of the interface layers.

2.3 Validation Architecture

Input validation is partitioned across three layers, each responsible for a distinct class of invariant:

Table 1: Validation responsibility by layer.

Layer	Module	Responsibility
Domain rules	`validators.py`	Node name format, weight range, NaN/Inf rejection, self-loop prevention
Structural	`graph.py`	Node existence, auto-creation of endpoints, adjacency consistency
UI/interaction	`pygame_view.py`	Scroll-wheel guard, click blocking during input, decimal deduplication, surface `ValueError` messages to the user

3 Core Data Structures

3.1 Graph Representation

Definition 3.1 (Weighted Undirected Graph)**.** A weighted undirected graph is a triple $G = (V, E, w)$ where V is a finite set of vertices, $E \subseteq \{\{u, v\} : u, v \in V,\ u \neq v\}$ is the edge set, and $w : E \to \mathbb{R}_{\geq 0}$ is the weight function.

RouteForge represents G using an *adjacency list* stored as a nested Python dictionary:

$$\texttt{\_adj} : \text{str} \ \to \ (\text{str} \ \to \ \text{float})$$

Each outer key is a node name; its value is a dictionary mapping neighbour names to edge weights. Every undirected edge $\{u, v, w\}$ is stored as two directed entries: `_adj[u][v] = w` and `_adj[v][u] = w`.

3.1.1 Choice of Adjacency List

An adjacency matrix would offer $O(1)$ edge lookup but requires $O(|V|^2)$ space and is poorly suited to dynamic graphs. The adjacency list provides:

Table 2: Time complexity of core `Graph` operations.

Operation	Complexity	Notes
`add_node`	$O(1)$	Dict insert
`add_edge`	$O(1)$	Two dict inserts + validation
`remove_node`	$O(\deg(v))$	Must remove reverse entries
`remove_edge`	$O(1)$	Two dict deletes
`has_node`	$O(1)$	Dict membership
`has_edge`	$O(1)$	Nested dict membership
`neighbors`	$O(\deg(v))$	List of (neighbour, weight)
`edges`	$O(\|V\| + \|E\|)$	Deduplication via canonical key
Space	$O(\|V\| + \|E\|)$	

3.2 Edge Dataclass

Edges are returned as instances of an immutable, hashable `Edge` dataclass:

```
@dataclass(frozen=True)
class Edge:
    u: str
    v: str
    weight: float
```

Listing 1: Edge dataclass definition.

The `frozen=True` flag makes `Edge` instances immutable and hashable, enabling their use in sets and as dictionary keys. The `edges()` method deduplicates using a canonical key $(\min(u, v), \max(u, v))$, ensuring each undirected edge appears exactly once regardless of the order in which endpoints are passed to `add_edge`.

3.3 Position Storage

The `Graph` class maintains a secondary dictionary `positions : str → (int, int)` mapping node names to pixel coordinates. This coupling is a pragmatic design choice: keeping positions on the graph object means the visualiser receives a fully self-contained graph when the CLI passes it to `GraphVisualizer`, with no need for a separate coordinate store. The `remove_node` method clears the associated position entry to prevent stale data.

3.4 Disjoint Set (Union-Find)

Kruskal's algorithm requires a disjoint-set data structure to detect cycles efficiently. The `DisjointSet` class in `kruskal.py` implements union by rank with path compression:

```python
class DisjointSet:
    def __init__(self, items):
        self.parent = {item: item for item in items}
        self.rank   = {item: 0    for item in items}

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, a, b):
        root_a, root_b = self.find(a), self.find(b)
        if root_a == root_b:
            return False                                 # same component
        if self.rank[root_a] < self.rank[root_b]:
            self.parent[root_a] = root_b
        elif self.rank[root_a] > self.rank[root_b]:
            self.parent[root_b] = root_a
        else:
            self.parent[root_b] = root_a
            self.rank[root_a]  += 1
        return True
```

Listing 2: Disjoint-Set with path compression and union by rank.

With path compression and union by rank, the amortised cost of m union-find operations on n elements is $O(m\,\alpha(n))$, where α is the inverse Ackermann function, effectively constant for all practical inputs.

4 Input Validation

4.1 Overview

The `validators.py` module centralises all domain-level input rules. Every entry point that mutates the graph (`add_node`, `add_edge`) calls the appropriate validator before any structural change occurs. The UI layer calls `validate_weight` independently so it can display a precise error message before invoking `add_edge`.

4.2 Node Name Validation

`validate_node_name(name)` enforces:

- Non-empty string.

- Maximum length of 16 characters.
- Characters restricted to `[a-zA-Z0-9_-]`.

4.3 Weight Validation

`validate_weight(weight)` enforces:

- Must be a numeric type (`int` or `float`).
- Must not be `NaN` or $\pm\infty$.
- Must lie within $[0.0,\ 10^6]$.

4.4 Edge Validation

`validate_edge(u, v, weight)` is a convenience wrapper that calls `validate_edge_nodes` (which validates both node names and enforces no self-loops) followed by `validate_weight`.

All validators raise `ValueError` with descriptive messages on failure. The UI layer catches these exceptions and renders the message in the status bar, ensuring no crash occurs and the user receives actionable feedback.

5 Breadth-First Search

5.1 Algorithm Description

Breadth-First Search explores a graph level by level, visiting all neighbours of a node before proceeding to their neighbours. It uses a FIFO queue to maintain the frontier.

Algorithm 1 Breadth-First Search

```
function BFS(G, start)
    visited ← {start}
    queue ← ⟨start⟩
    order ← []
    while queue ≠ ∅ do
        node ← DEQUEUE(queue)
        order.APPEND(node)
        for each (neighbour, _) in SORTEDNEIGHBOURS(G, node) do
            if neighbour ∉ visited then
                visited.ADD(neighbour)
                ENQUEUE(queue, neighbour)
            end if
        end for
    end whilereturn order
end function
```

5.2 Implementation Notes

The implementation uses `collections.deque` for $O(1)$ `popleft` operations (a Python `list` used as a queue would give $O(n)$ `pop(0)`). Neighbours are sorted lexicographically to produce a deterministic traversal order, which is important for testability.

```python
def bfs(graph: Graph, start: str) -> List[str]:
    if not graph.has_node(start):
        raise KeyError(f"Start node '{start}' does not exist.")

    visited = set([start])
    queue   = deque([start])
    order: List[str] = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbor, _ in sorted(graph.neighbors(node), key=lambda
           x: x[0]):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order
```

Listing 3: BFS implementation (`src/algorithms/bfs.py`).

5.3 Complexity Analysis

Table 3: BFS complexity.

	Time	Space
BFS	$O(\|V\| + \|E\|)$	$O(\|V\|)$

Each node is enqueued and dequeued at most once ($O(|V|)$). Each edge is examined at most twice ($O(|E|)$). The `visited` set and queue together hold at most $O(|V|)$ elements.

6 Depth-First Search

6.1 Algorithm Description

Depth-First Search explores as far as possible along each branch before backtracking. The implementation uses an explicit stack (iterative DFS) to avoid Python's recursion depth limit.

Algorithm 2 Depth-First Search (Iterative)

```
1: function DFS(G, start)
2:     visited ← ∅
3:     stack ← [start]
4:     order ← []
5:     while stack ≠ ∅ do
6:         node ← stack.POP()
7:         if node ∈ visited then
8:             continue
9:         end if
10:        visited.ADD(node)
11:        order.APPEND(node)
12:        for each (neighbour, _) in REVERSESORTEDNEIGHBOURS(G, node) do
13:            if neighbour ∉ visited then
14:                stack.PUSH(neighbour)
15:            end if
16:        end for
17:    end whilereturn order
18: end function
```

6.2 Implementation Notes

Neighbours are pushed in reverse-sorted order so that the lexicographically smallest neighbour is on top of the stack and explored first, matching the traversal order of a recursive DFS. The visited check on `pop` (rather than on `push`) is correct for iterative DFS: a node may be pushed multiple times before it is first visited, so the check must occur at the point of expansion.

```python
def dfs(graph: Graph, start: str) -> List[str]:
    if not graph.has_node(start):
        raise KeyError(f"Start node '{start}' does not exist.")

    visited = set()
    stack   = [start]
    order: List[str] = []

    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)

        neighbors = sorted(graph.neighbors(node),
                           key=lambda x: x[0], reverse=True)
        for neighbor, _ in neighbors:
            if neighbor not in visited:
                stack.append(neighbor)

    return order
```

Listing 4: DFS implementation (`src/algorithms/dfs.py`).

6.3 Complexity Analysis

Table 4: DFS complexity.

	Time	Space
DFS	$O(\|V\| + \|E\|)$	$O(\|V\|)$

7 Dijkstra's Shortest-Path Algorithm

7.1 Algorithm Description

Dijkstra's algorithm computes single-source shortest paths in a graph with non-negative edge weights. It maintains a min-priority queue of (distance, node) pairs and greedily expands the closest unvisited node.

Algorithm 3 Dijkstra's Algorithm

function DIJKSTRA(G, $start$)
    $dist[v] \leftarrow \infty$ for all $v \in V$; $dist[start] \leftarrow 0$
    $prev[v] \leftarrow$ NONE for all $v \in V$
    $pq \leftarrow \{(0, start)\}$
    $visited \leftarrow \emptyset$
    while $pq \neq \emptyset$ **do**
        $(d, node) \leftarrow$ EXTRACTMIN(pq)
        if $node \in visited$ **then**
            continue
        end if
        $visited$.ADD($node$)
        for each $(neighbour, weight)$ in Neighbours$(G, node)$ **do**
            $d' \leftarrow d + weight$
            if $d' < dist[neighbour]$ **then**
                $dist[neighbour] \leftarrow d'$
                $prev[neighbour] \leftarrow node$
                pq.INSERT$(d', neighbour)$
            end if
        end for
    end whilereturn $dist$, $prev$
end function

7.2 Path Reconstruction

The `reconstruct_path` helper in `helpers.py` traverses the predecessor map backwards from the end node, collecting nodes until the start is reached, then reverses the list:

```
def reconstruct_path(previous, start, end):
    path, current = [], end
    while current is not None:
        path.append(current)
        if current == start:
            break
```

```
        current = previous.get(current)
    if not path or path[-1] != start:
        return None
    path.reverse()
    return path
```

Listing 5: Path reconstruction (`src/algorithms/helpers.py`).

If the end node is unreachable, `previous[end]` will remain `None` and the traversal will terminate without reaching `start`, so `None` is returned.

7.3 Implementation Notes

The implementation uses Python's `heapq` module, which provides a binary min-heap. Lazy deletion is used: when a shorter path to a node is found, a new entry is pushed onto the heap without removing the old entry. Stale entries are discarded by the check for settled vertices.

```
def dijkstra(graph, start):
    distances = {node: float("inf") for node in graph.nodes()}
    previous  = {node: None         for node in graph.nodes()}
    distances[start] = 0.0
    pq       = [(0.0, start)]
    visited = set()

    while pq:
        current_dist, node = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph.neighbors(node):
            new_dist = current_dist + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                previous[neighbor]  = node
                heapq.heappush(pq, (new_dist, neighbor))

    return distances, previous
```

Listing 6: Dijkstra core (`src/algorithms/dijkstra.py`).

7.4 Complexity Analysis

With a binary heap and an adjacency list:

Table 5: Dijkstra complexity.

	Time	**Space**
Dijkstra (binary heap)	$O((\|V\| + \|E\|) \log \|V\|)$	$O(\|V\|)$

With lazy deletion, up to $O(|E|)$ entries may reside in the heap simultaneously, giving a heap size of $O(|E|)$ and each extraction costing $O(\log |E|) = O(\log |V|)$ for sparse graphs.

7.5 Correctness Condition

Theorem 7.1. *Dijkstra's algorithm is correct for graphs with non-negative edge weights.*

The invariant is maintained throughout: when a node u is extracted from the priority queue, $dist[u]$ holds the true shortest-path distance from *start* to u. This is guaranteed because all edge weights are non-negative, so no future relaxation can produce a shorter path to an already settled node. The `validators.py` weight check ($w \geq 0$) enforces this precondition at graph construction time.

8 Minimum Spanning Tree Algorithms

8.1 Prim's Algorithm

8.1.1 Description

Prim's algorithm grows the MST one edge at a time by repeatedly selecting the minimum-weight edge that connects a visited node to an unvisited node. It is a greedy algorithm whose correctness follows from the Cut Property of MSTs.

Algorithm 4 Prim's MST

```
function PrimMST(G, start)
    visited ← {start}
    mst ← []
    pq ← ∅
    for each (neighbour, w) in Neighbours(G, start) do
        pq.Insert(w, start, neighbour)
    end for
    while pq ≠ ∅ and |visited| < |V| do
        (w, u, v) ← ExtractMin(pq)
        if v ∈ visited then
            continue
        end if
        visited.Add(v)
        mst.Append(Edge(u, v, w))
        for each (neighbour, wt) in Neighbours(G, v) do
            if neighbour ∉ visited then
                pq.Insert(wt, v, neighbour)
            end if
        end for
    end whilereturn mst
end function
```

8.1.2 Complexity

Table 6: Prim's algorithm complexity (binary heap).

	Time	**Space**
Prim (binary heap)	$O(\|E\| \log \|V\|)$	$O(\|V\| + \|E\|)$

8.2 Kruskal's Algorithm

8.2.1 Description

Kruskal's algorithm processes all edges in ascending weight order, adding each edge to the MST if and only if it connects two previously disconnected components. Component tracking is delegated to the `DisjointSet` structure described in Section **??**.

Algorithm 5 Kruskal's MST

```
function KruskalMST(G)
    ds ← DisjointSet(V)
    edges ← SortedByWeight(E)
    mst ← []
    for each e = (u, v, w) in edges do
        if Union(ds, u, v) then
            mst.Append(e)
            if |mst| = |V| − 1 then
                break
            end if
        end if
    end forreturn mst
end function
```

8.2.2 Early Termination

The loop breaks as soon as $|V| - 1$ edges have been accepted, since a spanning tree on $|V|$ nodes has exactly $|V| - 1$ edges. This avoids processing the remaining (necessarily rejected) edges.

8.2.3 Complexity

Table 7: Kruskal's algorithm complexity.

	Time	**Space**
Kruskal	$O(\|E\| \log \|E\|)$	$O(\|V\|)$

The dominant cost is sorting the edges: $O(|E| \log |E|)$. Each union-find operation is effectively $O(1)$ amortised.

8.3 Comparison of MST Algorithms

Table 8: Prim vs. Kruskal comparison.

Property	Prim	Kruskal
Approach	Vertex-centric, grows one connected component	Edge-centric, merges components
Data structure	Min-heap	Disjoint-set + sorted edge list
Preferred for	Dense graphs ($\|E\| \approx \|V\|^2$)	Sparse graphs ($\|E\| \approx \|V\|$)
Requires	Connected graph (from start)	Works on forests
Start node	Required (defaults to first node)	Not required

9 Graph Visualiser

9.1 Overview

The visualiser (`src/visualization/pygame_view.py`) is a self-contained pygame application that supports two primary modes:

Edit mode The user constructs a graph interactively: left-clicking empty canvas space creates a node; clicking two nodes in sequence initiates an edge, with weight entered via a floating on-canvas input box.

Algorithm mode BFS, DFS, or Dijkstra is animated step-by-step with full playback controls.

9.2 Interaction State Machine

The visualiser maintains a string-valued mode variable with four states:

Table 9: Visualiser mode states.

State	Trigger	Description
`"edit"`	Initial / Reset (`R`)	Graph editing enabled
`"pick_start"`	`B`, `D`, or `K`	Awaiting start node click
`"pick_end"`	Start node chosen (Dijkstra only)	Awaiting end node click
`"algo"`	Both nodes chosen	AlgoPlayer active

Transitions between states are one-directional except for the global `Escape`/`R` reset that returns unconditionally to `"edit"`.

9.3 Animation Model

The `AlgoStep` dataclass captures a complete snapshot of the algorithm state at one point in execution:

```
@dataclass
class AlgoStep:
    visited     : Set[str]
    frontier    : Set[str]
    active_node : Optional[str]
    active_edges: Set[Tuple[str, str]]
    active_edge : Optional[Tuple[str, str]]
    dist_labels : Dict[str, str]          # Dijkstra only
    path_nodes  : Set[str]                # Dijkstra final step
    path_edges  : Set[Tuple[str, str]]    # Dijkstra final step
    description : str
```

Listing 7: AlgoStep dataclass.